Penalized Regression

Kachun Huang

2025-04-09

Penalized Regression in R

Import libraries

```
library(glmnet)
```

```
## Loading required package: Matrix
```

```
## Loaded glmnet 4.1-8
```

```
library(caret)
```

```
## Warning: package 'caret' was built under R version 4.3.3
```

```
## Loading required package: ggplot2
```

```
## Loading required package: lattice
```

```
## Warning: package 'lattice' was built under R version 4.3.3
```

```
library(MASS)
library(tidyverse)
```

```
## Warning: package 'purrr' was built under R version 4.3.3
```

```
## Warning: package 'lubridate' was built under R version 4.3.3
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.5
## v forcats   1.0.0     v stringr   1.5.1
## v lubridate 1.9.4     v tibble    3.2.1
## v purrr     1.0.4     v tidyr     1.3.1
```

```
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x tidyr::expand() masks Matrix::expand()
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## x purrr::lift()   masks caret::lift()
## x tidyr::pack()   masks Matrix::pack()
## x dplyr::select() masks MASS::select()
## x tidyr::unpack() masks Matrix::unpack()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become error
```

Model Matrix

```
x <- model.matrix(response ~., train.data)[,-1]
y <- train.data$response
```

Ridge Regression

Ridge regression adds a penalty equal to the square of the magnitude of coefficients, which shrinks the coefficients towards zero but does not set them exactly to zero:

```
# Find the best lambda using cross-validation
cv.ridge <- cv.glmnet(x, y, alpha = 0)
```

Best Lambda

```
best_lambda <- cv.ridge$lambda.min
```

Fit the Model with best lambda

```
# Fit the model with the best lambda
ridge_model <- glmnet(x, y, alpha = 0, lambda = best_lambda)
```

```
coef(ridge_model)
```

Display coefficients

Lasso Regression

Lasso regression uses the L1 penalty, which can shrink some coefficients to exactly zero, effectively performing variable selection:

```
# Find the best lambda using cross-validation
cv.lasso <- cv.glmnet(x, y, alpha = 1)
```

Best Lambda

```
best_lambda <- cv.lasso$lambda.min
```

Fit the Model with best lambda

```r
# Fit the model with the best lambda
lasso_model <- glmnet(x, y, alpha = 1, lambda = best_lambda)
```

```r
coef(lasso_model)
```

Display coefficients

Elastic Net Regression

Elastic Net combines the properties of both Ridge and Lasso by using both L1 and L2 penalties:

We need to find the optimal α and λ

```r
model <- train(
  repsonse~.,
  data = train.data,
  method = "glmnet",
  trControl = trainControl("cv", number=10),
  tuneLength = 10
)
```

Best Alpha and Lambda

```r
model$bestTune
```

Fit the Model with best lambda

```r
# Fit the model with the best lambda and alpha
elastic_model <- glmnet(x, y, alpha = best_alpha, lambda = best_lambda)
```

```r
coef(elastic_model)
```

Display coefficients

Example

Import Data

```r
data("Boston", package="MASS")
str(Boston)
```

```
## 'data.frame':    506 obs. of  14 variables:
##  $ crim   : num  0.00632 0.02731 0.02729 0.03237 0.06905 ...
##  $ zn     : num  18 0 0 0 0 0 12.5 12.5 12.5 12.5 ...
##  $ indus  : num  2.31 7.07 7.07 2.18 2.18 2.18 7.87 7.87 7.87 7.87 ...
##  $ chas   : int  0 0 0 0 0 0 0 0 0 0 ...
##  $ nox    : num  0.538 0.469 0.469 0.458 0.458 0.458 0.524 0.524 0.524 0.524 ...
##  $ rm     : num  6.58 6.42 7.18 7 7.15 ...
##  $ age    : num  65.2 78.9 61.1 45.8 54.2 58.7 66.6 96.1 100 85.9 ...
##  $ dis    : num  4.09 4.97 4.97 6.06 6.06 ...
##  $ rad    : int  1 2 2 3 3 3 5 5 5 5 ...
##  $ tax    : num  296 242 242 222 222 222 311 311 311 311 ...
##  $ ptratio: num  15.3 17.8 17.8 18.7 18.7 18.7 15.2 15.2 15.2 15.2 ...
##  $ black  : num  397 397 393 395 397 ...
##  $ lstat  : num  4.98 9.14 4.03 2.94 5.33 ...
##  $ medv   : num  24 21.6 34.7 33.4 36.2 28.7 22.9 27.1 16.5 18.9 ...
```

Data Splitting

```r
set.seed(123)
training.samples <- Boston$medv %>% createDataPartition(p=0.75, list=FALSE)
train.data <- Boston[training.samples,]
test.data <- Boston[-training.samples,]
```

Model Matrix

```r
x <- model.matrix(medv~., train.data)[,-1] #exclude the last column (medv)
y <- train.data$medv
```

Fit Ridge Regression

```r
cv <- cv.glmnet(
 x,
 y,
 alpha=0
)
```

Best Lambda

```r
cv$lambda.min
```

```
## [1] 0.6490823
```

Fit the Model with best lambda

```
model <- glmnet(
  x,
  y,
  alpha=0,
  lambda = cv$lambda.min
)
```

```
coef(model)
```

Model Coefficient

```
## 14 x 1 sparse Matrix of class "dgCMatrix"
##                         s0
## (Intercept)  25.994735184
## crim         -0.066393301
## zn            0.018062258
## indus        -0.058721267
## chas          1.738033967
## nox         -11.213169927
## rm            4.092962823
## age          -0.003856820
## dis          -0.849835447
## rad           0.118962860
## tax          -0.006818129
## ptratio      -0.842092676
## black         0.007931751
## lstat        -0.385975629
```

Prediction

```
x.test <- model.matrix(medv~., test.data)[,-1]
pred_ridge <- model %>% predict(x.test) %>% as.vector()
```

RMSE and R^2

```
data.frame(
  RMSE = RMSE(pred_ridge, test.data$medv),
  Rsquare = R2(pred_ridge, test.data$medv)
)
```

```
##       RMSE   Rsquare
## 1 6.635525 0.6626213
```

Fit Lasso Regression

```
cv <- cv.glmnet(
 x,
 y,
 alpha=1
)
```

Best Lambda

```
cv$lambda.min
```

```
## [1] 0.02943509
```

Fit the Model with best lambda

```
model <- glmnet(
  x,
  y,
  alpha=1,
  lambda = cv$lambda.min
)
```

```
coef(model)
```

Model Coefficient

```
## 14 x 1 sparse Matrix of class "dgCMatrix"
##                        s0
## (Intercept)  30.737740248
## crim         -0.070051875
## zn            0.023056204
## indus        -0.013502700
## chas          1.444465893
## nox         -14.898532459
## rm            4.083374808
## age           .
## dis          -1.028888982
## rad           0.210742993
## tax          -0.011072116
## ptratio      -0.917788678
## black         0.007918102
## lstat        -0.424325830
```

Prediction

```
x.test <- model.matrix(medv~., test.data)[,-1]
pred_lasso <- model %>% predict(x.test) %>% as.vector()
```

RMSE and R^2

```
data.frame(
  RMSE = RMSE(pred_lasso, test.data$medv),
  Rsquare = R2(pred_lasso, test.data$medv)
)
```

```
##       RMSE   Rsquare
## 1 6.472275 0.6749717
```

Fit Elastic Net Regression

```
model <- train(
  medv~.,
  data=train.data,
  method="glmnet",
  trControl=trainControl("cv",number=10),
  tuneLength=10
)
```

Best Alpha and Lambda

```
model$bestTune
```

```
##    alpha      lambda
## 72   0.8 0.006927914
```

Fit the Model with best lambda and alpha

```
model <- glmnet(
  x,
  y,
  alpha=model$bestTune$alpha,
  lambda=model$bestTune$lambda
)
```

```
coef(model)
```

Model Coefficient

```
## 14 x 1 sparse Matrix of class "dgCMatrix"
##                          s0
## (Intercept)  32.271751093
## crim         -0.077865969
## zn            0.026625737
## indus        -0.011923547
## chas          1.483662560
## nox         -15.808707289
## rm            4.037114756
## age           .
## dis          -1.104280767
## rad           0.248895635
## tax          -0.012686888
## ptratio      -0.931697576
## black         0.008117568
## lstat        -0.424713058
```

Prediction

```
x.test <- model.matrix(medv~., test.data)[,-1]
pred_en <- model %>% predict(x.test) %>% as.vector()
```

RMSE and R^2

```
data.frame(
  RMSE = RMSE(pred_en, test.data$medv),
  Rsquare = R2(pred_en, test.data$medv)
)
```

```
##       RMSE   Rsquare
## 1 6.431165 0.6782381
```

Ridge Regression using train()

```
set.seed(123)

grid_ridge <- expand.grid(
  alpha=0,
  lambda = seq(0.001,0.1, length=100)
)
```

```
model_ridge <- train(
  medv~.,
  data=train.data,
  method="glmnet",
  metric="RMSE",
  trConrol=trainControl(method="cv",number=10),
  tuneGrid=grid_ridge
)
```

Best Tune

```
model_ridge$bestTune
```

```
##     alpha lambda
## 100     0    0.1
```

```
coef(model_ridge)
```

Model Coefficient

```
## NULL
```

Prediction

```
x.test <- model.matrix(medv~., test.data)[,-1]
pred_ridge <- model_ridge %>% predict(x.test) %>% as.vector()
```

RMSE and R^2

```
data.frame(
  RMSE = RMSE(pred_ridge, test.data$medv),
  Rsquare = R2(pred_ridge, test.data$medv)
)
```

```
##       RMSE   Rsquare
## 1 6.634721 0.6627636
```

Lasso Regression using train()

```r
set.seed(123)

grid_lasso <- expand.grid(
  alpha=1,
  lambda = seq(0.001,0.1, length=100)
)

model_lasso <- train(
  medv~.,
  data=train.data,
  method="glmnet",
  metric="RMSE",
  trConrol=trainControl(method="cv",number=10),
  tuneGrid=grid_lasso
)
```

Best Tune

```r
model_lasso$bestTune
```

```
##    alpha lambda
## 17     1  0.017
```

```r
coef(model_lasso)
```

Model Coefficient

```
## NULL
```

Prediction

```r
x.test <- model.matrix(medv~., test.data)[,-1]
pred_lasso <- model_lasso %>% predict(x.test) %>% as.vector()
```

RMSE and R^2

```r
data.frame(
  RMSE = RMSE(pred_lasso, test.data$medv),
  Rsquare = R2(pred_lasso, test.data$medv)
)
```

```
##       RMSE   Rsquare
## 1 6.449878 0.6768284
```